

MagicCode Super-Resolution Software Known Issues

Document Version: v2.0.0. Release Date: September 2, 2026.

1. Scope

Known limitations for MagicCode Super-Resolution Software v2.0.0, including temporal super-resolution.

2. Functional constraints

- Input width/height must be 64 to 4032.
- Spatial scale [1, 8]; temporal (1, 8]. Not every scale is device-run.
- CPU backends have no temporal path.
- MC_Enable cannot supply temporal extras. Passing a temporal enum still does not create a complete temporal session.
- Derived reactive/transparency are approximations. Missing caller transparency is not a semantic mask; FSR-family paths bind zero transparency when omitted.

Temporal Super-Resolution

v2.0.0 adds temporal super-resolution through the session API: libmagic_sr.a (Windows: libmagic_sr.lib) + mc_interface.h + MC_Init / MC_Process / MC_Uninit. MC_Enable remains a single-texture spatial helper. It may accept a temporal alg_mode_e value by range check, but it cannot supply per-frame depth, motion, or jitter and must not be documented as a complete temporal entry.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGLS currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced edge-resolve is supported on Windows Vulkan, Windows Direct3D 11, desktop

OpenGL 4.3, and Apple Metal. Android Vulkan and Android OpenGL ES do not run edge-resolve. If `enable_sharpening` is on, RCAS runs after edge-resolve.

- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2x has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- `magic_frame_t.image_in / image_out`: caller-owned color and output GPU resources.
- `magic_frame_t.frame`: non-NULL `temporal_frame_t*` with `struct_size = sizeof(temporal_frame_t)`.
- `temporal_frame_t.depth` and motion at input resolution.
- `jitter_offset_x/y` in input pixels, top-left, +X right, +Y down.
- `frame_index / reset_history`; `camera_near / camera_far / camera_fov_y` (radians).
- `gpu_context` at `MC_Init`: Vulkan requires `physical_device + device`; D3D11 requires device; Metal device may be NULL; GL/GLES use the current context (library never makes current).
- Vulkan temporal requires `magic_frame_t.command_buffer` to be a recording `VkCommandBuffer`. Metal `command_buffer` is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage `depth_reversed / depth_infinite / hdr_color` on `input_param_t` (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- `motion_vector_scale_x/y` is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: `MAGICSR_TAAU_AUTO_MASK=0` and `MAGICSR_TAAU_HIST_REJECT=0` can turn those defaults off. They are not a customer configuration path.

3. Backend constraints

- Vulkan temporal: recording command buffer required; UNDEFINED layout rejected; no library-owned `VkDevice`.
- D3D11 temporal: `gpu_context.device` required. `Enable_4params` does not accept `MAGIC_BACKEND_D3D11`.
- GL/GLES: library never makes a context current.
- Vendor/driver differences on Android Vulkan/GLES and Windows D3D11/Vulkan/GL remain.

4. Engine plugins

- Unity/UE public Enable APIs are spatial. Full temporal is Native session API.
- Unreal iOS packaging still depends on matching UE/Xcode/SDK versions.

5. Operational

- Ghosting if reset_history is not set on camera cuts or if MV/jitter contracts are wrong (current→previous, +Y down).
- frame_index != last+1 is a warning, not a hard fail; history may be stale.
- Do not use diagnostic environment kill switches as a product settings UI.

6. Mitigation

- Validate each target platform with production-like motion and depth.
- Use QUERY_STATUS and MC_ERROR_TEMPORAL_* codes.
- Keep Enable for single-texture spatial; keep session API for temporal.